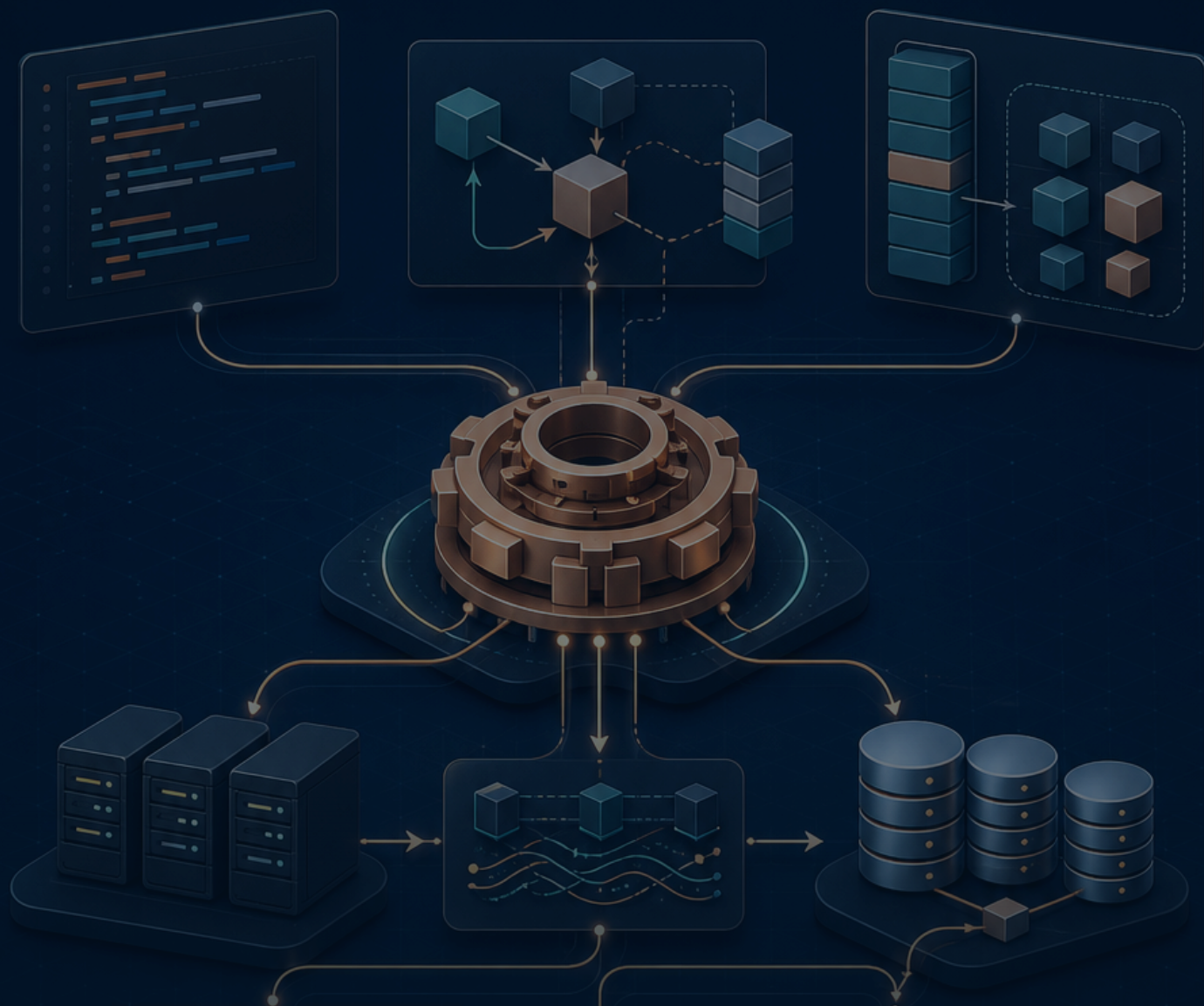




VOLUME 04 / RUST BACKEND MASTERY

Rust設計 — 型・trait・エラー・Cargo・テスト

動けばよいコードから、変更理由と失敗境界が型・module・testで読めるバックエンド設計へ進む

10

深掘り単元

20+

実行・解答例

40+

図・表・画面

実務

設計と障害対応

LEARNING MAP

この巻で身に付けること

newtype、trait、generic、エラー分類、Cargo features、DTO、設定、状態機械を一つの設計言語として結びます。

到達目標

- ✓ 不正状態を作れない型を設計できる
- ✓ traitで外部依存を反転できる
- ✓ エラーを分類してHTTP境界へ変換できる
- ✓ module・feature・testで公開契約を守れる

学習地図

01 newtypeと不正状態を作れない型

同じu64でもUserIdとNotIdは意味が違います。tuple structで型を分け、生成関数で範囲や形式を検証すると、引数の取り違えと未検証値の流入をコンパイル時に減らせます。

02 traitと境界による設計

traitは複数の型が満たす振る舞いの契約です。ユースケースがDB実装そのものではなくRepository traitへ依存すると、Turso実装とテスト用Fakeを交換できます。

03 generics・where句・静的ディスパッチ

genericsは型を引数化し、同じ処理を型安全に再利用します。trait boundで必要な操作だけを要求し、where句で複雑な制約を読みやすくします。通常は単相化され高速です。

04 dyn Traitとオブジェクト安全性

実行時に異なる実装を一つのコレクションやフィールドへ入れるときdyn Traitを使います。vtableによる動的ディスパッチとヒープ間接化の費用があるため、必要な境界に限定します。

05 エラー型・thiserror相当の設計・文脈

エラーは利用者向け表示ではなく、失敗原因と回復方針をプログラムへ伝える型です。Validation、NotFound、Forbidden、Conflict、Unavailableを区別し、HTTP変換は外側の層で行います。

06 crate・module・可視性・workspace

crateはコンパイル単位、moduleは名前空間と可視性境界、packageはCargo.toml単位、workspaceは複数packageを一つの依存解決とtargetで管理します。pubを広げすぎないことが変更耐性を上げます。

07 Cargo features・依存固定・最小構成

featureは条件付きコンパイルで能力を選びます。default featureの無自覚な有効化はビルド時間、バイナリ、攻撃面を増やすため、必要な機能だけを明示し、アプリではCargo.lockを固定します。

08 DTO・Serde・境界の変換

外部JSON、フォーム、DB行、ドメイン型を一つのstructで兼用すると、未検証値や秘密フィールドが漏れます。Deserializeする入力DTO、検証済みコマンド、Serializeする出力DTOを分けます。

09 設定型・環境変数・秘密

環境変数はすべて文字列で、未設定・空・不正形式を起動時に検証する必要があります。Config型を一度作り、その後のコードからenv::varを直接呼ばないようにします。

10 テスト・rustdoc・状態機械・APIレビュー

設計の意図はテストとドキュメントへ固定します。状態遷移をenumとメソッドで表し、正常系、禁止遷移、境界値をテストします。公開APIには例と失敗条件を書きます。

HOW TO USE THIS VOLUME

読むだけで終わらせない進め方

「予想→実行→観察→一か所変更→回帰テスト」を一単位ごとに繰り返します。写経した行数ではなく、入力・処理・出力・失敗を自分の言葉で説明できるかを進捗にします。

1. 予想

実行前に入力型、出力、失敗箇所を予想します。

2. そのまま実行

最初は変更せず、教材と同じ出力が比較します。

3. 一か所変更

値または型を一つだけ変え、差分の理由を説明します。

4. 回帰テスト

直した失敗を自動テストへ残し、再発を防ぎます。

STEP 1

意味ごとの型分離

STEP 2

private field

STEP 3

検証済み生成

STEP 4

Displayと変換

毎回そろえる確認コマンド

● ● ● 単元を終える前の品質確認

```
cargo fmt --check
cargo clippy --all-targets -- -D warnings
cargo test --locked --all-targets
```

対象	この教材の基準	混在を防ぐ理由
Rust	2024 Edition / rust-version 1.95以上	editionとMSRVを明示し、環境差を再現可能にする
Topcoat	crate / CLI ともに 0.5.0固定	early-stageのため、mainブランチの別構文を混ぜない
DB・認証	Turso/libSQL、Argon2id、Topcoat Session	一つのRustプロセスで責務とデータの流れを追える
公開	Docker + Fly.io、HOST=0.0.0.0、PORT=8080	ローカルとコンテナの待受差を明示する

重要な版の前提 Topcoatは公式がearly-stage / experimentalと明記しています。本教材はcrateとCLIを **0.5.0** に固定し、mainブランチの別構文を混ぜません。

UNIT 01

newtypeと不正状態を作れない型

UNIT 01 · CONCEPT

newtypeと不正状態を作れない型

同じu64でもUserIdとNotelIdは意味が違います。tuple structで型を分け、生成関数で範囲や形式を検証すると、引数の取り違えと未検証値の流入をコンパイル時に減らせます。

この単元の到達点

- 意味ごとの型分離
- private field
- 検証済み生成
- Displayと変換

なぜバックエンドで必要か

path parameterをNotelIdへ変換してからユースケースへ渡すと、負数やUserIdとの取り違えを境界で止められます。



UNIT 01 · MENTAL MODEL

newtypeと不正状態を作れない型を図でつかむ



同じu64でもUserIdとNotelIdは意味が違います。tuple structで型を分け、生成関数で範囲や形式を検証すると、引数の取り違えと未検証値の流入をコンパイル時に減らせます。図では左から右へ処理を追います。各箱の入口では入力 の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	意味ごとの型分離	意味ごとの型分離 — 入力と前提を確認し、境界を曖昧にしない。
2	private field	private field — 値がどの順で変換されるかを追跡する。
3	検証済み生成	検証済み生成 — 失敗を再現し、型またはログから原因を特定する。
4	Displayと変換	Displayと変換 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

newtypeと不正状態を作れない型とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 01・MINIMUM CODE

まず動く最小コード

newtypeと不正状態を作れない型だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
struct NoteId(i64);
impl NoteId {
    fn new(value: i64) -> Result<Self, &'static str> {
        if value > 0 { Ok(Self(value)) } else { Err("id must be positive") }
    }
}
fn main() { println!("{:?}", NoteId::new(7)); }
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 01・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[derive(Debug, Clone, Copy, PartialEq, Eq)]</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>struct NoteId(i64);</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>impl NoteId {</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>fn new(value: i64) -> Result<Self, &'static str> {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>if value > 0 { Ok(Self(value)) } else { Err("id must be positive") }</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力が意味ごとの型分離の条件を満たすか確認する。
- ✓ `private field`を実行し、途中の型と状態を追う。
- ✓ 検証済み生成で失敗を成功経路から分離する。
- ✓ `Display`と変換により、出力と副作用を検証する。

型を声に出す `newtype`と不正状態を作れない型に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 01 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

`Ok(NoteId(7))`

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 01 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
#[derive(Debug, PartialEq)]
struct EmailAddress(String);
impl EmailAddress {
    fn parse(raw: &str) -> Result<Self, &'static str> {
        let value = raw.trim().to_lowercase();
        if value.contains('@') && value.len() <= 254 { Ok(Self(value)) } else { Err("invalid email") }
    }
}
fn main() { assert_eq!(EmailAddress::parse(" A@EXAMPLE.COM "), Ok(EmailAddress("a@example.com".into()))); }
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 01・BACKEND APPLICATION

バックエンドのどこで使うか

path parameterをNoteldへ変換してからユースケースへ渡すと、負数やUserIdとの取り違えを境界で止められます。



境界で守る不変条件

- ✓ 不正な入力副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 01・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	意味ごとの型分離に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	private fieldの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	検証済み生成またはDisplayと変換の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 01・DEBUGGING

newtypeと不正状態を作れない型を再現して切り分ける

newtypeと不正状態を作れない型の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: 意味ごとの型分離
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: private field
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: 検証済み生成
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: Displayと変換

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	意味ごとの型分離	意味ごとの型分離 — 入力と前提を確認し、境界を曖昧にしない。
2	private field	private field — 値がどの順で変換されるかを追跡する。
3	検証済み生成	検証済み生成 — 失敗を再現し、型またはログから原因を特定する。
4	Displayと変換	Displayと変換 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 意味と検証状態を型へ持たせ、primitive obsessionを避けます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 01 · PRACTICE

手を動かす演習

EmailAddress newtypeを作り、trim・小文字化・@の存在確認を生成時に行ってください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 01 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
#[derive(Debug, PartialEq)]
struct EmailAddress(String);
impl EmailAddress {
    fn parse(raw: &str) -> Result<Self, &'static str> {
        let value = raw.trim().to_lowercase();
        if value.contains('@') && value.len() <= 254 { Ok(Self(value)) } else { Err("invalid email") }
    }
}
fn main() { assert_eq!(EmailAddress::parse(" A@EXAMPLE.COM "), Ok(EmailAddress("a@example.com".into()))); }
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 意味と検証状態を型へ持たせ、primitive obsessionを避けます。
- ✓ newtypeと不正状態を作れない型とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 02

traitと境界による設計

UNIT 02 · CONCEPT

traitと境界による設計

traitは複数の型が満たす振る舞いの契約です。ユースケースがDB実装そのものではなくRepository traitへ依存すると、Turso実装とテスト用Fakeを交換できます。

この単元の到達点

- 振る舞いの契約
- trait impl
- 依存性逆転
- テストダブル

なぜバックエンドで必要か

時刻、ID生成、メール送信、Repositoryをtrait化すると、外部状態を決定的なFakeへ置き換えられます。



UNIT 02 · MENTAL MODEL

traitと境界による設計を図でつかむ



traitは複数の型が満たす振る舞いの契約です。ユースケースがDB実装そのものではなくRepository traitへ依存すると、Turso実装とテスト用Fakeを交換できます。図では左から右へ処理を追います。各箱の入口では入力型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	振る舞いの契約	振る舞いの契約 — 入力と前提を確認し、境界を曖昧にしない。
2	trait impl	trait impl — 値がどの順で変換されるかを追跡する。
3	依存性逆転	依存性逆転 — 失敗を再現し、型またはログから原因を特定する。
4	テストダブル	テストダブル — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

traitと境界による設計とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 02 · MINIMUM CODE まず動く最小コード

traitと境界による設計だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
trait Clock { fn now_epoch(&self) -> i64; }
struct FixedClock(i64);
impl Clock for FixedClock { fn now_epoch(&self) -> i64 { self.0 } }
fn created_label(clock: &impl Clock) -> String { format!("created_at={}", clock.now_epoch()) }
fn main() { println!("{}", created_label(&FixedClock(123))); }
```

●●● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 02 · LINE BY LINE コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>trait Clock { fn now_epoch(&self) -> i64; }</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>struct FixedClock(i64);</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>impl Clock for FixedClock { fn now_epoch(&self) -> i64 { self.0 } }</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>fn created_label(clock: &impl Clock) -> String { format!("created_at={}",</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>fn main() { println!("{}", created_label(&FixedClock(123))); }</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。

実行時に起きる順序

- ✓ 入力振る舞いの契約の条件を満たすか確認する。
- ✓ `trait impl`を実行し、途中の型と状態を追う。
- ✓ 依存性逆転で失敗を成功経路から分離する。
- ✓ テストダブルにより、出力と副作用を検証する。

型を声に出す `trait`と境界による設計に関する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 02 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

`created_at=123`

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 02 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
trait NoteRepository { fn save(&mut self, title: String) -> u64; fn find(&self, id: u64) -> Option<&str>; }
struct MemoryRepo { rows: Vec<String> }
impl NoteRepository for MemoryRepo {
    fn save(&mut self, title: String) -> u64 { self.rows.push(title); self.rows.len() as u64 }
    fn find(&self, id: u64) -> Option<&str> { self.rows.get(id.checked_sub(1)? as usize).map(String::as_str) }
}
fn main() { let mut repo=MemoryRepo{rows:vec![]}; let id=repo.save("Rust".into()); assert_eq!(
    repo.find(id),Some("Rust")); }
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 02 ・ BACKEND APPLICATION

バックエンドのどこで使うか

時刻、ID生成、メール送信、Repositoryをtrait化すると、外部状態を決定的なFakeへ置き換えられます。



リクエスト側

- 振る舞いの契約を入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- trait implを小さな責務へ分割
- 依存性逆転をResultとログで表現
- テストダブルをテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 02 ・ FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	振る舞いの契約に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	trait implの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	依存性逆転またはテストダブルの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 02・DEBUGGING

traitと境界による設計を再現して切り分ける

traitと境界による設計の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: 振る舞いの契約
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: trait impl
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: 依存性逆転
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: テストダブル

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	振る舞いの契約	振る舞いの契約 — 入力と前提を確認し、境界を曖昧にしない。
2	trait impl	trait impl — 値がどの順で変換されるかを追跡する。
3	依存性逆転	依存性逆転 — 失敗を再現し、型またはログから原因を特定する。
4	テストダブル	テストダブル — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 ユースケースは必要な振る舞いをtraitで要求し、外部実装の詳細から切り離します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 02 ・ PRACTICE

手を動かす演習

NoteRepository traitとInMemoryNoteRepositoryを作り、create後にfindできるテストを書いてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 02 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
trait NoteRepository { fn save(&mut self, title: String) -> u64; fn find(&self, id: u64) -> Option<&str>; }
struct MemoryRepo { rows: Vec<String> }
impl NoteRepository for MemoryRepo {
    fn save(&mut self, title: String) -> u64 { self.rows.push(title); self.rows.len() as u64 }
    fn find(&self, id: u64) -> Option<&str> { self.rows.get(id.checked_sub(1)? as usize).map(String::as_str) }
}
fn main() { let mut repo=MemoryRepo{rows:vec![]}; let id=repo.save("Rust".into()); assert_eq!(
repo.find(id),Some("Rust")); }
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ ユースケースは必要な振る舞いをtraitで要求し、外部実装の詳細から切り離します。
- ✓ traitと境界による設計とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 03

generics・where句・静的ディスパッチ

UNIT 03・CONCEPT

generics・where句・静的ディスパッチ

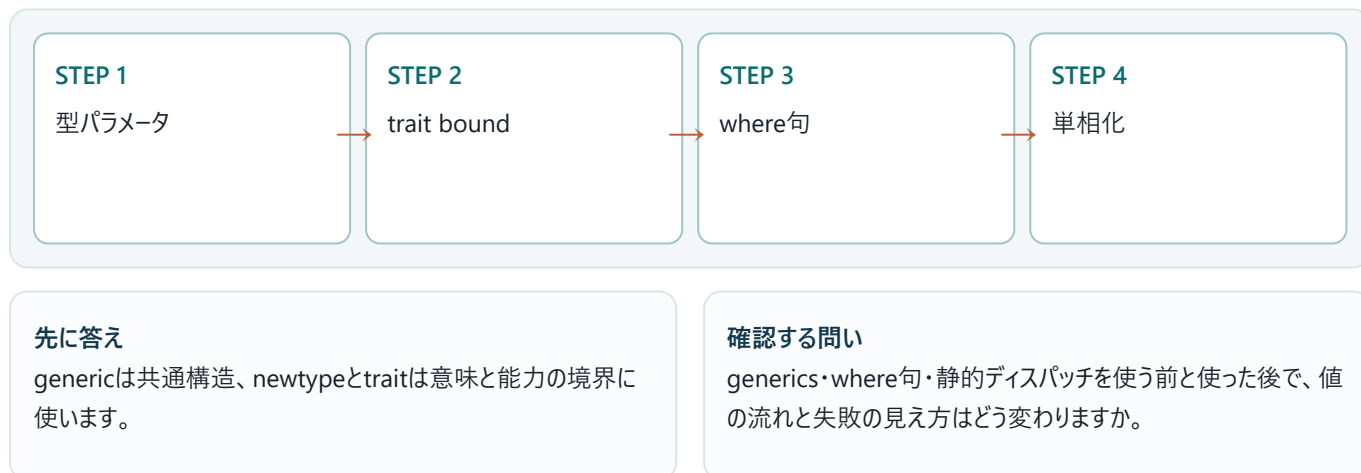
genericsは型を引数化し、同じ処理を型安全に再利用します。trait boundで必要な操作だけを要求し、where句で複雑な制約を読みやすくします。通常は単相化され高速です。

この単元の到達点

- 型パラメータ
- trait bound
- where句
- 単相化

なぜバックエンドで必要か

共通のページング結果、キャッシュ、入力パーサーをgenericにしつつ、ドメイン固有の検証は専用型へ残します。



UNIT 03・MENTAL MODEL

generics・where句・静的ディスパッチを図でつかむ



genericsは型を引数化し、同じ処理を型安全に再利用します。trait boundで必要な操作だけを要求し、where句で複雑な制約を読みやすくします。通常は単相化され高速です。図では左から右へ処理を追います。各箱の入口では入力型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	型パラメータ	型パラメータ — 入力と前提を確認し、境界を曖昧にしない。
2	trait bound	trait bound — 値がどの順で変換されるかを追跡する。
3	where句	where句 — 失敗を再現し、型またはログから原因を特定する。
4	単相化	単相化 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

generics・where句・静的ディスパッチとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 03・MINIMUM CODE まず動く最小コード

generics・where句・静的ディスパッチだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
use std::fmt::Display;
fn audit<T>(kind: &str, value: T) -> String
where T: Display {
    format!("kind={kind} value={value}")
}
fn main() { println!("{}", audit("note_id", 42)); }
```

●●● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 03・LINE BY LINE コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>use std::fmt::Display;</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>fn audit<T>(kind: &str, value: T) -> String</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>where T: Display {</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>format!("kind={kind} value={value}")</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>}</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力型パラメータの条件を満たすか確認する。
- ✓ `trait bound`を実行し、途中の型と状態を追う。
- ✓ `where`句で失敗を成功経路から分離する。
- ✓ 単相化により、出力と副作用を検証する。

型を声に出す `generics`・`where`句・静的ディスパッチに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 03 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

`kind=note_id value=42`

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 03 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
#[derive(Debug, PartialEq)] struct Page<T> { items: Vec<T>, page: u32, total: u64 }
impl<T> Page<T> {
    fn map<U>(self, f: impl FnMut(T) -> U) -> Page<U> { Page { items:
self.items.into_iter().map(f).collect(), page: self.page, total: self.total } }
}
fn main() { let p=Page{items:vec![1,2],page:1,total:2}; assert_eq!(p.map(|n| n.to_string()).items, vec!
["1","2"]); }
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

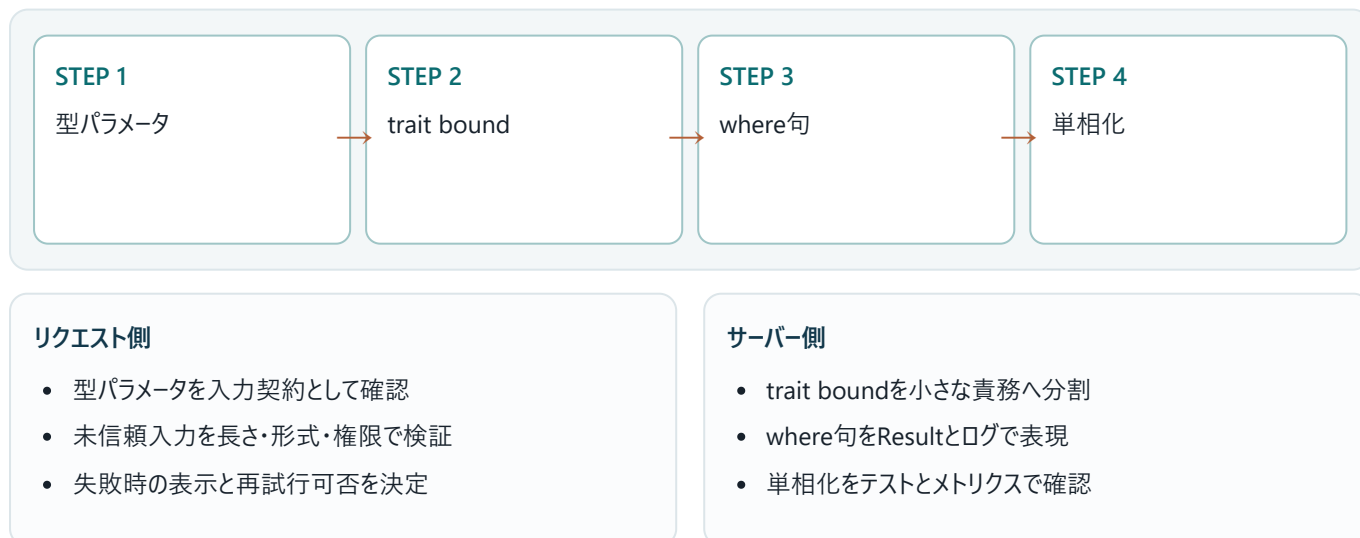
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 03 · BACKEND APPLICATION

バックエンドのどこで使うか

共通のページング結果、キャッシュ、入力パーサーをgenericにしつつ、ドメイン固有の検証は専用型へ残します。



境界で守る不変条件

- ✓ 不正な入力副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 03・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	型パラメータに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	trait boundの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	where句または単相化の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 03・DEBUGGING

generics・where句・静的ディスパッチを再現して切り分ける

generics・where句・静的ディスパッチの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: 型パラメータ
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: trait bound
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: where句
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 単相化

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	型パラメータ	型パラメータ — 入力と前提を確認し、境界を曖昧にしない。
2	trait bound	trait bound — 値がどの順で変換されるかを追跡する。
3	where句	where句 — 失敗を再現し、型またはログから原因を特定する。
4	単相化	単相化 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 genericは共通構造、newtypeとtraitは意味と能力の境界に使います。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 03 ・ PRACTICE

手を動かす演習

Page<T>を定義し、items、page、totalを持たせ、mapでPage<T>からPage<U>へ変換してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 03 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
#[derive(Debug, PartialEq)] struct Page<T> { items: Vec<T>, page: u32, total: u64 }
impl<T> Page<T> {
    fn map<U>(self, f: impl FnMut(T) -> U) -> Page<U> { Page { items:
self.items.into_iter().map(f).collect(), page: self.page, total: self.total } }
}
fn main() { let p=Page{items:vec![1,2],page:1,total:2}; assert_eq!(p.map(|n| n.to_string()).items, vec!
["1", "2"]); }
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ genericは共通構造、newtypeとtraitは意味と能力の境界に使います。
- ✓ generics・where句・静的ディスパッチとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 04

dyn Traitとオブジェクト安全性

UNIT 04 · CONCEPT

dyn Traitとオブジェクト安全性

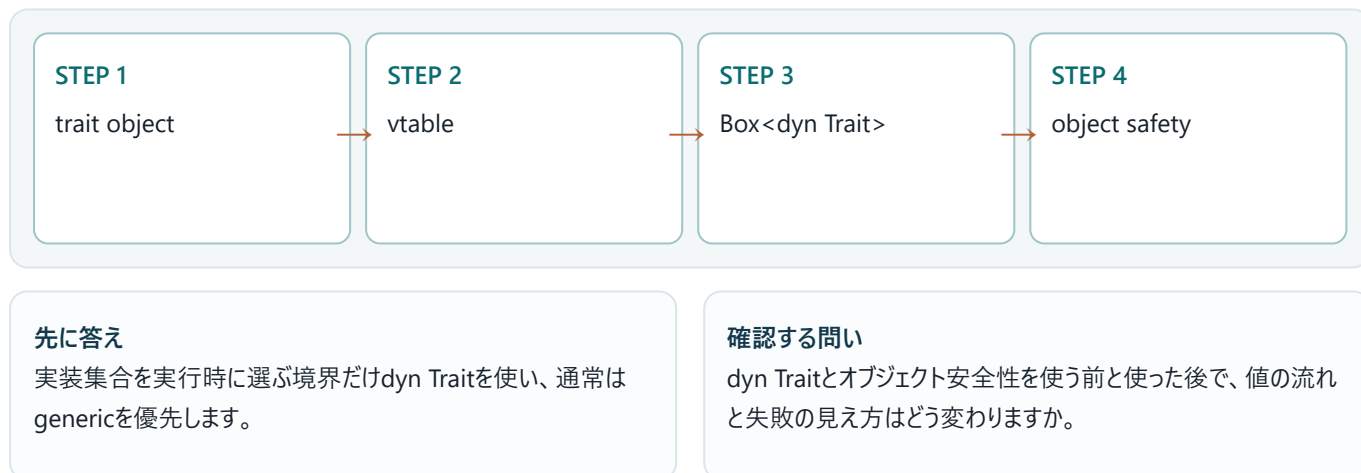
実行時に異なる実装を一つのコレクションやフィールドへ入れるときdyn Traitを使います。vtableによる動的ディスパッチとヒープ間接化の費用があるため、必要な境界に限定します。

この単元の到達点

- trait object
- vtable
- Box<dyn Trait>
- object safety

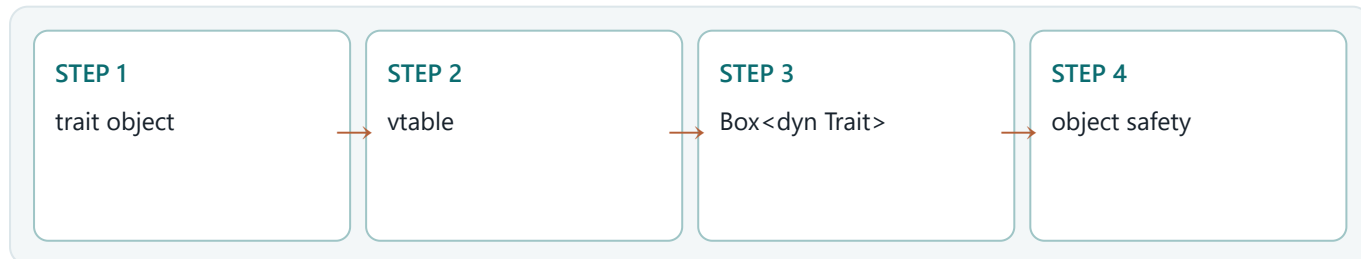
なぜバックエンドで必要か

通知チャネルや認証providerを設定で切り替える境界ではdyn Traitが有効です。hot pathではgenericとの費用差を測ります。



UNIT 04 · MENTAL MODEL

dyn Traitとオブジェクト安全性を図でつかむ



実行時に異なる実装を一つのコレクションやフィールドへ入れるときdyn Traitを使います。vtableによる動的ディスパッチとヒープ間接化の費用があるため、必要な境界に限定します。図では左から右へ処理を追います。各箱の入口では入力の種類と信頼度、出口では結果の種類と副作用を確認します。

順序	観察対象	初心者が確認すること
1	trait object	trait object — 入力と前提を確認し、境界を曖昧にしない。
2	vtable	vtable — 値がどの順で変換されるかを追跡する。
3	Box<dyn Trait>	Box<dyn Trait> — 失敗を再現し、型またはログから原因を特定する。
4	object safety	object safety — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

dyn Traitとオブジェクト安全性とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 04 · MINIMUM CODE まず動く最小コード

dyn Traitとオブジェクト安全性だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
trait Notifier { fn send(&self, text: &str) -> Result<(), String>; }
struct Console;
impl Notifier for Console { fn send(&self, text: &str) -> Result<(), String> { println!("notify: {text}");
Ok(()) } }
fn dispatch(notifiers: &[Box<dyn Notifier>]) { for n in notifiers { let _ = n.send("published"); } }
fn main() { dispatch(&[Box::new(Console)]); }
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 04 · LINE BY LINE コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>trait Notifier { fn send(&self, text: &str) -> Result<(), String>; }</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>struct Console;</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>impl Notifier for Console { fn send(&self, text: &str) -> Result<(), Strin</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>fn dispatch(notifiers: &[Box<dyn Notifier>]) { for n in notifiers { let _</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>fn main() { dispatch(&[Box::new(Console)]); }</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。

実行時に起きる順序

- ✓ 入力 trait object の条件を満たすか確認する。
- ✓ vtable を実行し、途中の型と状態を追う。
- ✓ Box<dyn Trait> で失敗を成功経路から分離する。
- ✓ object safety により、出力と副作用を検証する。

型を声に出す dyn Trait とオブジェクト安全性に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 04 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

notify: published

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTP ステータス、標準エラー、ログ、DB の行数など、画面に現れない観測点も合わせて確認します。

UNIT 04 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
trait Notifier { fn send(&self, text: &str); }
struct Console; impl Notifier for Console { fn send(&self, text:&str){println!("{text}")} }
struct Silent; impl Notifier for Silent { fn send(&self,&str){} }
fn main(){ let xs:Vec<Box<dyn Notifier>>=vec![Box::new(Console),Box::new(Silent)]; for x in xs{x.send("event");} }
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 04 · BACKEND APPLICATION

バックエンドのどこで使うか

通知チャンネルや認証providerを設定で切り替える境界ではdyn Traitが有効です。hot pathではgenericとの費用差を測ります。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 04 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	trait objectに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	vtableの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	Box<dyn Trait>またはobject safetyの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 04 · DEBUGGING

dyn Traitとオブジェクト安全性を再現して切り分ける

dyn Traitとオブジェクト安全性の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: trait object
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: vtable
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: Box<dyn Trait>
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: object safety

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	trait object	trait object — 入力と前提を確認し、境界を曖昧にしない。
2	vtable	vtable — 値がどの順で変換されるかを追跡する。
3	Box<dyn Trait>	Box<dyn Trait> — 失敗を再現し、型またはログから原因を特定する。
4	object safety	object safety — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 実装集合を実行時に選ぶ境界だけdyn Traitを使い、通常はgenericを優先します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 04 ・ PRACTICE

手を動かす演習

ConsoleNotifierとRecordingNotifierを同じVec<Box<dyn Notifier>>で呼び出してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 04 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
trait Notifier { fn send(&self, text: &str); }
struct Console; impl Notifier for Console { fn send(&self, text:&str){println!("{text}")} }
struct Silent; impl Notifier for Silent { fn send(&self, _:&str){} }
fn main(){ let xs:Vec<Box<dyn Notifier>>=vec![Box::new(Console),Box::new(Silent)]; for x in xs{x.send("event");} }
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 実装集合を実行時に選ぶ境界だけdyn Traitを使い、通常はgenericを優先します。
- ✓ dyn Traitとオブジェクト安全性とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 05

エラー型・thiserror相当の設計・文脈

UNIT 05 · CONCEPT

エラー型・thiserror相当の設計・文脈

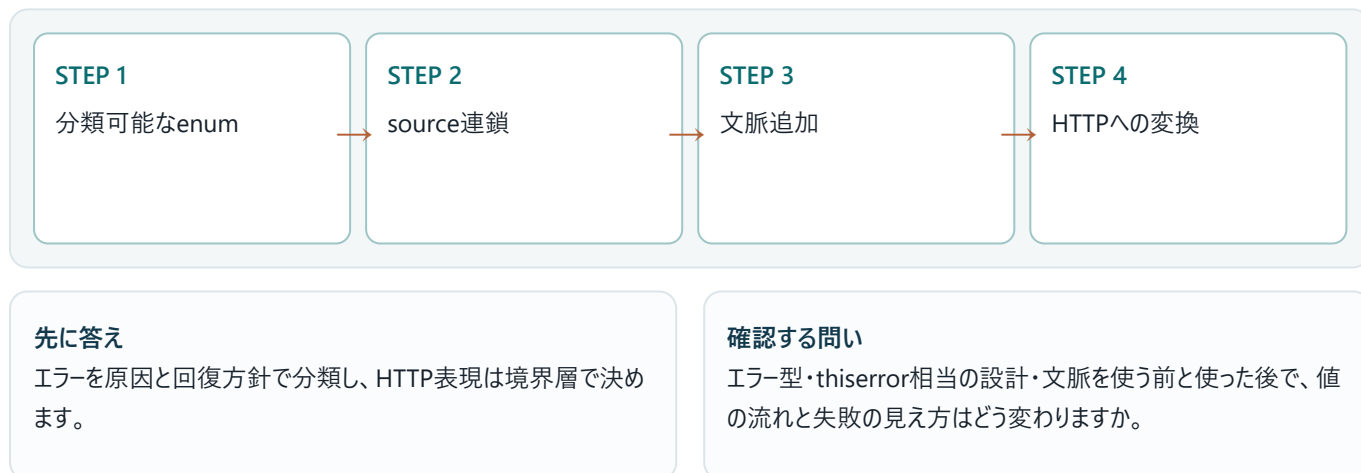
エラーは利用者向け表示ではなく、失敗原因と回復方針をプログラムへ伝える型です。Validation、NotFound、Forbidden、Conflict、Unavailableを区別し、HTTP変換は外側の層で行います。

この単元の到達点

- 分類可能なenum
- source連鎖
- 文脈追加
- HTTPへの変換

なぜバックエンドで必要か

内部エラーには原因連鎖とrequest_idを残し、レスポンスには秘密やSQLを出さず安定した分類だけを返します。



UNIT 05 · MENTAL MODEL

エラー型・thiserror相当の設計・文脈を図でつかむ



エラーは利用者向け表示ではなく、失敗原因と回復方針をプログラムへ伝える型です。Validation、NotFound、Forbidden、Conflict、Unavailableを区別し、HTTP変換は外側の層で行います。図では左から右へ処理を追います。各箱の入口では入力型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	分類可能なenum	分類可能なenum — 入力と前提を確認し、境界を曖昧にしない。
2	source連鎖	source連鎖 — 値がどの順で変換されるかを追跡する。
3	文脈追加	文脈追加 — 失敗を再現し、型またはログから原因を特定する。
4	HTTPへの変換	HTTPへの変換 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

エラー型・thiserror相当の設計・文脈とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 05 ・ MINIMUM CODE

まず動く最小コード

エラー型・thiserror相当の設計・文脈だけに注目した最小例です。まず変更せず実行し、出力が一致した後に一行ずつ意味を確認します。

```
#[derive(Debug)]
enum AppError { Validation(String), NotFound, Forbidden, Conflict }
fn status(error: &AppError) -> u16 {
    match error {
        AppError::Validation(_) => 400, AppError::Forbidden => 403,
        AppError::NotFound => 404, AppError::Conflict => 409,
    }
}
fn main(){ println!("{}", status(&AppError::NotFound)); }
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 05 ・ LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[derive(Debug)]</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>enum AppError { Validation(String), NotFound, Forbidden, Conflict }</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>fn status(error: &AppError) -> u16 {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>match error {</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>AppError::Validation(_) => 400, AppError::Forbidden => 403,</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力分類可能なenumの条件を満たすか確認する。
- ✓ source連鎖を実行し、途中の型と状態を追う。
- ✓ 文脈追加で失敗を成功経路から分離する。
- ✓ HTTPへの変換により、出力と副作用を検証する。

型を声に出す エラー型・thiserror相当の設計・文脈に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 05・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

404

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 05 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
#[derive(Debug)] enum AppError { Validation, NotFound, Unavailable { operation: &'static str } }
fn to_status(e:&AppError)->u16 { match e { AppError::Validation=>400, AppError::NotFound=>404,
AppError::Unavailable{..}>=>503 } }
fn load(simulate_failure:bool)->Result<(),AppError>{ if
simulate_failure{Err(AppError::Unavailable{operation:"load_note"})}else{Ok(())} }
fn main(){ assert_eq!(to_status(&load(true).unwrap_err()),503); }
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 05 ・ BACKEND APPLICATION

バックエンドのどこで使うか

内部エラーには原因連鎖とrequest_idを残し、レスポンスには秘密やSQLを出さず安定した分類だけを返します。



リクエスト側

- 分類可能なenumを入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- source連鎖を小さな責務へ分割
- 文脈追加をResultとログで表現
- HTTPへの変換をテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 05 ・ FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	分類可能なenumに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違ふ	source連鎖の入力または実行順序が想定と違ふ	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	文脈追加またはHTTPへの変換の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。

✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 05・DEBUGGING

エラー型・thiserror相当の設計・文脈を再現して切り分ける

エラー型・thiserror相当の設計・文脈の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: 分類可能なenum
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: source連鎖
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: 文脈追加
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: HTTPへの変換

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	分類可能なenum	分類可能なenum — 入力と前提を確認し、境界を曖昧にしない。
2	source連鎖	source連鎖 — 値がどの順で変換されるかを追跡する。
3	文脈追加	文脈追加 — 失敗を再現し、型またはログから原因を特定する。
4	HTTPへの変換	HTTPへの変換 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 エラーを原因と回復方針で分類し、HTTP表現は境界層で決めます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 05 ・ PRACTICE

手を動かす演習

DBエラーを直接文字列表示せず、`AppError::Unavailable`へ包んでHTTP 503へ変換してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 05 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
#[derive(Debug)] enum AppError { Validation, NotFound, Unavailable { operation: &'static str } }
fn to_status(e:&AppError)->u16 { match e { AppError::Validation=>400, AppError::NotFound=>404,
AppError::Unavailable{..}>=>503 } }
fn load(simulate_failure:bool)->Result<(),AppError>{ if
simulate_failure{Err(AppError::Unavailable{operation:"load_note"})}else{Ok(())} }
fn main(){ assert_eq!(to_status(&load(true).unwrap_err()),503); }
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ エラーを原因と回復方針で分類し、HTTP表現は境界層で決めます。
- ✓ エラー型・thiserror相当の設計・文脈とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 06

crate・module・可視性・workspace

UNIT 06・CONCEPT

crate・module・可視性・workspace

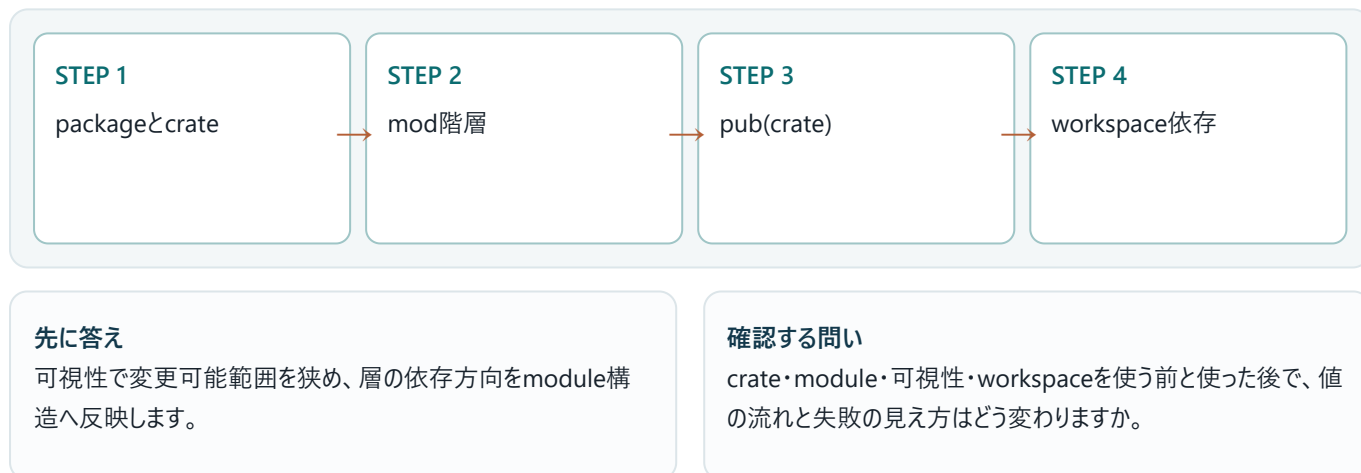
crateはコンパイル単位、moduleは名前空間と可視性境界、packageはCargo.toml単位、workspaceは複数packageを一つの依存解決とtargetで管理します。pubを広げすぎないことが変更耐性を上げます。

この単元の到達点

- packageとcrate
- mod階層
- pub(crate)
- workspace依存

なぜバックエンドで必要か

web、application、domain、infrastructureの依存方向をmodule可視性で守り、handlerからDB実装へ直接依存しない構造にします。



UNIT 06・MENTAL MODEL

crate・module・可視性・workspaceを図でつかむ



crateはコンパイル単位、moduleは名前空間と可視性境界、packageはCargo.toml単位、workspaceは複数packageを一つの依存解決とtargetで管理します。pubを広げすぎないことが変更耐性を上げます。図では左から右へ処理を追います。各箱の入口では入力 of 型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	packageとcrate	packageとcrate — 入力と前提を確認し、境界を曖昧にしない。
2	mod階層	mod階層 — 値がどの順で変換されるかを追跡する。
3	pub(crate)	pub(crate) — 失敗を再現し、型またはログから原因を特定する。
4	workspace依存	workspace依存 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

crate・module・可視性・workspaceとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 06・MINIMUM CODE
まず動く最小コード

crate・module・可視性・workspaceだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
// src/domain/mod.rs
pub(crate) mod note;
// src/domain/note.rs
#[derive(Debug)]
pub struct Note { pub id: i64, title: String }
impl Note { pub fn title(&self) -> &str { &self.title } }
// 外部からtitleを書き換えず、メソッド経由にする
```

●●● 実行コマンド

```
cargo check --all-targets
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 06・LINE BY LINE
コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>pub(crate) mod note;</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>#[derive(Debug)]</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>pub struct Note { pub id: i64, title: String }</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>impl Note { pub fn title(&self) -> &str { &self.title } }</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。

実行時に起きる順序

- ✓ 入力 package と crate の条件を満たすか確認する。
- ✓ mod 階層を実行し、途中の型と状態を追う。
- ✓ pub(crate) で失敗を成功経路から分離する。
- ✓ workspace 依存により、出力と副作用を検証する。

型を声に出す crate・module・可視性・workspace に関する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 06・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

●●● 実行結果

ドメイン型の内部フィールドは非公開、必要な読み取りだけメソッドで公開します。

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTP ステータス、標準エラー、ログ、DB の行数など、画面に現れない観測点も合わせて確認します。

UNIT 06・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
// src/lib.rs
pub mod web;
pub(crate) mod application;
pub(crate) mod infrastructure;
pub mod domain;

// domainはwebやinfrastructureをuseしない
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 06・BACKEND APPLICATION

バックエンドのどこで使うか

web、application、domain、infrastructureの依存方向をmodule可視性で守り、handlerからDB実装へ直接依存しない構造にします。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。

- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 06 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	packageとcrateに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	mod階層の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	pub(crate)またはworkspace依存の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 06・DEBUGGING

crate・module・可視性・workspaceを再現して切り分ける

crate・module・可視性・workspaceの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: packageとcrate
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: mod階層
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: pub(crate)
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: workspace依存

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	packageとcrate	packageとcrate — 入力と前提を確認し、境界を曖昧にしない。
2	mod階層	mod階層 — 値がどの順で変換されるかを追跡する。
3	pub(crate)	pub(crate) — 失敗を再現し、型またはログから原因を特定する。
4	workspace依存	workspace依存 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 可視性で変更可能範囲を狭め、層の依存方向をmodule構造へ反映します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 06 ・ PRACTICE

手を動かす演習

domain、application、infrastructure、webの4moduleを作り、依存方向をweb→application→domainにしてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 06 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
// src/lib.rs
pub mod web;
pub(crate) mod application;
pub(crate) mod infrastructure;
pub mod domain;

// domainはwebやinfrastructureをuseしない
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 可視性で変更可能範囲を狭め、層の依存方向をmodule構造へ反映します。
- ✓ crate・module・可視性・workspaceとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 07

Cargo features・依存固定・最小構成

UNIT 07・CONCEPT

Cargo features・依存固定・最小構成

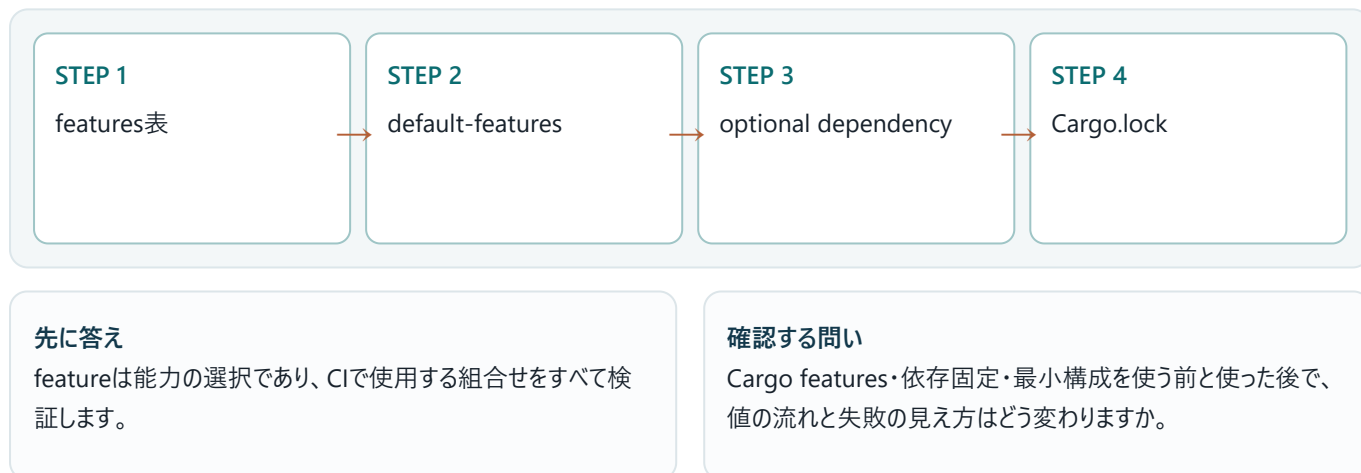
featureは条件付きコンパイルで能力を選びます。default featureの無自覚な有効化はビルド時間、バイナリ、攻撃面を増やすため、必要な機能だけを明示し、アプリではCargo.lockを固定します。

この単元の到達点

- features表
- default-features
- optional dependency
- Cargo.lock

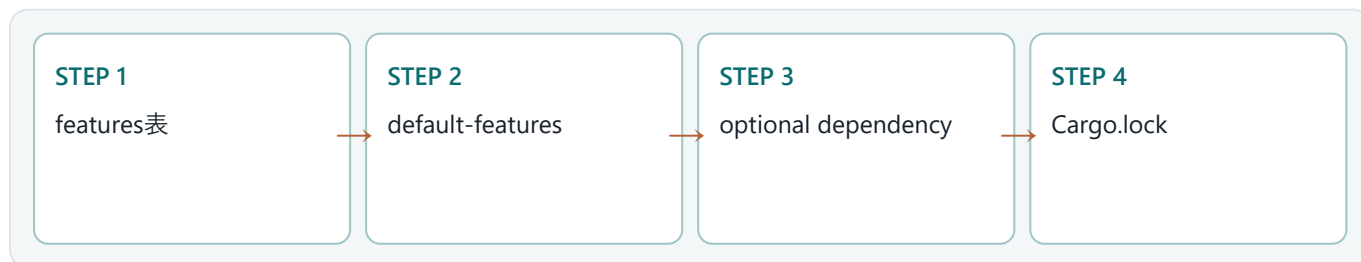
なぜバックエンドで必要か

本番とテストのfeature差をCIで検証し、使っていないTLSやDB driverを無効化して依存面を小さくします。



UNIT 07・MENTAL MODEL

Cargo features・依存固定・最小構成を図でつかむ



featureは条件付きコンパイルで能力を選びます。default featureの無自覚な有効化はビルド時間、バイナリ、攻撃面を増やすため、必要な機能だけを明示し、アプリではCargo.lockを固定します。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	features表	features表 — 入力と前提を確認し、境界を曖昧にしない。
2	default-features	default-features — 値がどの順で変換されるかを追跡する。
3	optional dependency	optional dependency — 失敗を再現し、型またはログから原因を特定する。
4	Cargo.lock	Cargo.lock — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出来されるか」を自分の言葉で説明します。

30秒説明

Cargo features・依存固定・最小構成とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 07・MINIMUM CODE
まず動く最小コード

Cargo features・依存固定・最小構成だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
[features]
default = []
telemetry = ["dep:tracing-subscriber"]

[dependencies]
serde = { version = "1", features = ["derive"] }
tracing-subscriber = { version = "0.3", optional = true }
```

●●● 実行コマンド

```
cargo check --no-default-features
cargo check --features telemetry
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 07・LINE BY LINE
コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
[features]	この行が受け取る値、作る値、副作用の有無を確認します。
default = []	この行が受け取る値、作る値、副作用の有無を確認します。
telemetry = ["dep:tracing-subscriber"]	この行が受け取る値、作る値、副作用の有無を確認します。
[dependencies]	この行が受け取る値、作る値、副作用の有無を確認します。
serde = { version = "1", features = ["derive"] }	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力 features 表の条件を満たすか確認する。
- ✓ default-features を実行し、途中の型と状態を追う。
- ✓ optional dependency で失敗を成功経路から分離する。
- ✓ Cargo.lock により、出力と副作用を検証する。

型を声に出す Cargo features・依存固定・最小構成に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値の一つを変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 07 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
cargo check --no-default-features
cargo check --features telemetry
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 07 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
#[cfg(feature = "telemetry")] fn init_logging(){ println!("telemetry on"); }
#[cfg(not(feature = "telemetry"))] fn init_logging(){ println!("telemetry off"); }
fn main(){ init_logging(); }
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

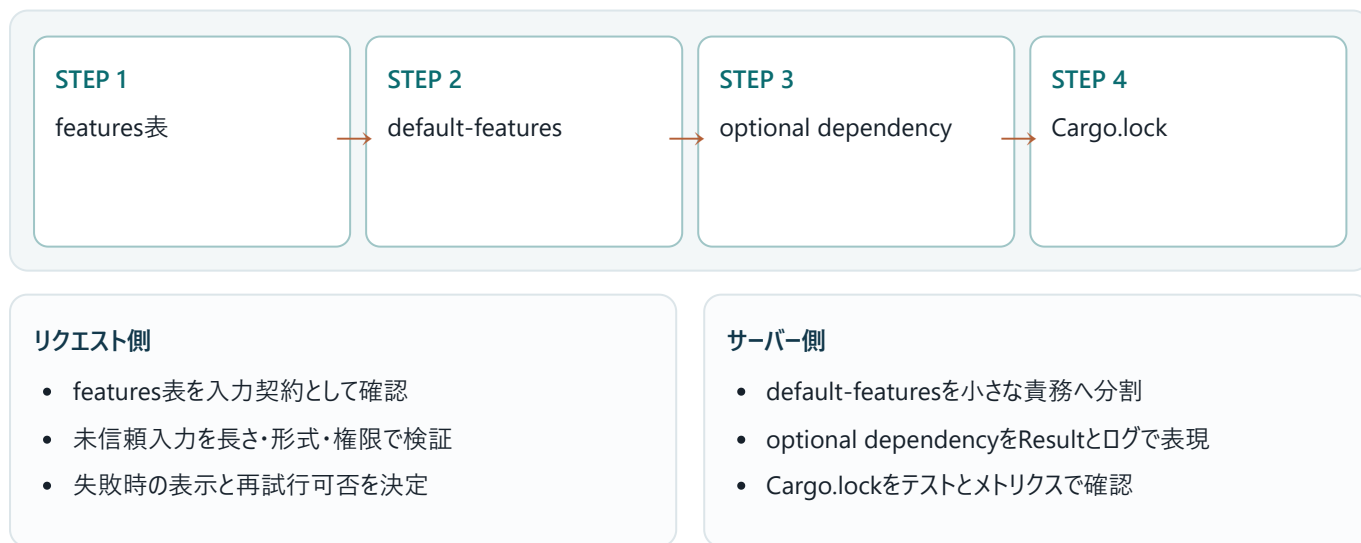
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 07 · BACKEND APPLICATION

バックエンドのどこで使うか

本番とテストのfeature差をCIで検証し、使っていないTLSやDB driverを無効化して依存面を小さくします。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。

- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 07 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	features表に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	default-featuresの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	optional dependencyまたはCargo.lockの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 07・DEBUGGING

Cargo features・依存固定・最小構成を再現して切り分ける

Cargo features・依存固定・最小構成の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: features表
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: default-features
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: optional dependency
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: Cargo.lock

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	features表	features表 — 入力と前提を確認し、境界を曖昧にしない。
2	default-features	default-features — 値がどの順で変換されるかを追跡する。
3	optional dependency	optional dependency — 失敗を再現し、型またはログから原因を特定する。
4	Cargo.lock	Cargo.lock — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 featureは能力の選択であり、CIで使用する組合せをすべて検証します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 07 · PRACTICE

手を動かす演習

telemetry feature有無でinit_logging関数の実装を切り替え、両方をcargo checkしてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 07 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
#[cfg(feature = "telemetry")] fn init_logging(){ println!("telemetry on"); }
#[cfg(not(feature = "telemetry"))] fn init_logging(){ println!("telemetry off"); }
fn main(){ init_logging(); }
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ featureは能力の選択であり、CIで使用する組合せをすべて検証します。
- ✓ Cargo features・依存固定・最小構成とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 08

DTO・Serde・境界の変換

UNIT 08・CONCEPT

DTO・Serde・境界の変換

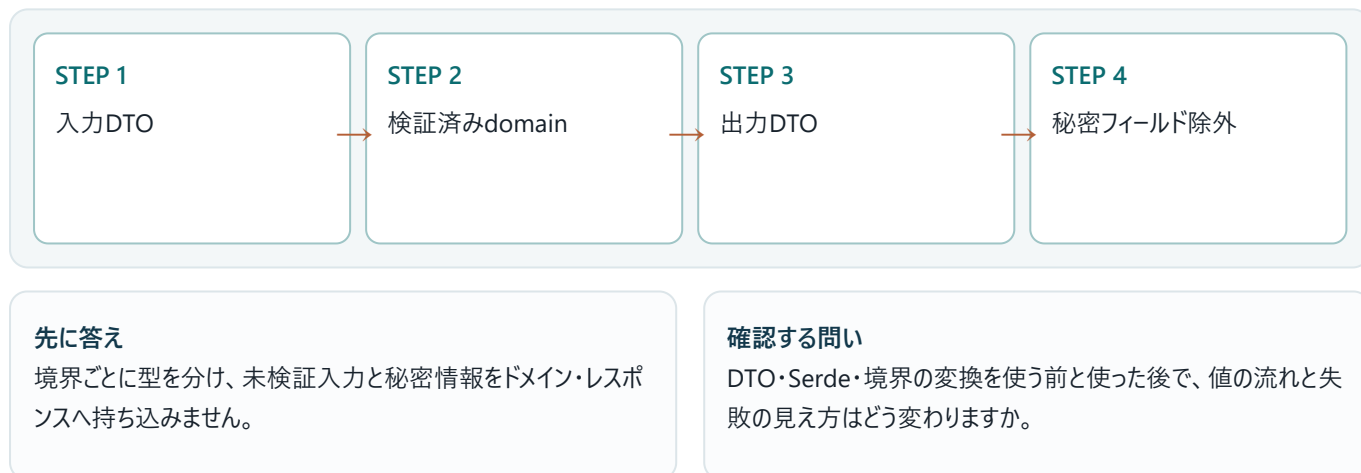
外部JSON、フォーム、DB行、ドメイン型を一つのstructで兼用すると、未検証値や秘密フィールドが漏れます。Deserializeする入力DTO、検証済みコマンド、Serializeする出力DTOを分けます。

この単元の到達点

- 入力DTO
- 検証済みdomain
- 出力DTO
- 秘密フィールド除外

なぜバックエンドで必要か

Web入力DTO、検証済みドメイン型、DB Row、レスポンスDTOを分けることで、未検証値と秘密フィールドの層越えを防ぎます。



UNIT 08・MENTAL MODEL

DTO・Serde・境界の変換を図でつかむ



外部JSON、フォーム、DB行、ドメイン型を一つのstructで兼用すると、未検証値や秘密フィールドが漏れます。Deserializeする入力DTO、検証済みコマンド、Serializeする出力DTOを分けます。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	入力DTO	入力DTO — 入力と前提を確認し、境界を曖昧にしない。
2	検証済みdomain	検証済みdomain — 値がどの順で変換されるかを追跡する。
3	出力DTO	出力DTO — 失敗を再現し、型またはログから原因を特定する。
4	秘密フィールド除外	秘密フィールド除外 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

DTO・Serde・境界の変換とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 08・MINIMUM CODE まず動く最小コード

DTO・Serde・境界の変換だけに注目した最小例です。まず変更せず実行し、出力が一致した後に一行ずつ意味を確認します。

```
use serde::{Deserialize, Serialize};
#[derive(Deserialize)] struct CreateNoteInput { title: String }
#[derive(Serialize)] struct NoteResponse { id: i64, title: String }
fn validate(input: CreateNoteInput) -> Result<String, &'static str> {
    let title=input.title.trim(); if title.is_empty(){Err("required")}else{Ok(title.into())}
}
```

●●● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 08・LINE BY LINE コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>use serde::{Deserialize, Serialize};</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>#[derive(Deserialize)] struct CreateNoteInput { title: String }</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>#[derive(Serialize)] struct NoteResponse { id: i64, title: String }</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>fn validate(input: CreateNoteInput) -> Result<String, &'static str> {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>let title=input.title.trim(); if title.is_empty() {Err("required")}else{Ok(</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。

実行時に起きる順序

- ✓ 入力が入力DTOの条件を満たすか確認する。
- ✓ 検証済みdomainを実行し、途中の型と状態を追う。
- ✓ 出力DTOで失敗を成功経路から分離する。
- ✓ 秘密フィールド除外により、出力と副作用を検証する。

型を声に出す DTO・Serde・境界の変換に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 08 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● http://localhost:3000/learn

Rust Backend Lab

実行する

DTO・Serde・境界の変換

JSONレスポンスにはidとemailだけが表示され、password_hashは型に存在しないため漏れません。

観察ポイント

- 入力DTO
- 検証済みdomain
- 出力DTO

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 08・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
struct UserRow { id:i64, email:String, password_hash:String }
#[derive(serde::Serialize)] struct UserResponse { id:i64, email:String }
impl From<UserRow> for UserResponse { fn from(row:UserRow)->Self{ Self{id:row.id,email:row.email} } }
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける

観点	最小例	実務版
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 08・BACKEND APPLICATION

バックエンドのどこで使うか

Web入力DTO、検証済みドメイン型、DB Row、レスポンスDTOを分けることで、未検証値と秘密フィールドの層越えを防ぎます。



よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	入力DTOに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	検証済みdomainの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	出力DTOまたは秘密フィールド除外の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 08・DEBUGGING

DTO・Serde・境界の変換を再現して切り分ける

DTO・Serde・境界の変換の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: 入力DTO
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: 検証済みdomain
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: 出力DTO
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 秘密フィールド除外

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	入力DTO	入力DTO — 入力と前提を確認し、境界を曖昧にしない。
2	検証済みdomain	検証済みdomain — 値がどの順で変換されるかを追跡する。
3	出力DTO	出力DTO — 失敗を再現し、型またはログから原因を特定する。
4	秘密フィールド除外	秘密フィールド除外 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 境界ごとに型を分け、未検証入力と秘密情報をドメイン・レスポンスへ持ち込みません。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 08 ・ PRACTICE

手を動かす演習

DataRowにpassword_hashがあってもUserResponseへ含まれない別型設計を作ってください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 08 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
struct UserRow { id:i64, email:String, password_hash:String }
#[derive(serde::Serialize)] struct UserResponse { id:i64, email:String }
impl From<UserRow> for UserResponse { fn from(row:UserRow)->Self{ Self{id:row.id,email:row.email} } }
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 境界ごとに型を分け、未検証入力と秘密情報をドメイン・レスポンスへ持ち込みません。
- ✓ DTO・Serde・境界の変換とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 09

設定型・環境変数・秘密

UNIT 09・CONCEPT

設定型・環境変数・秘密

環境変数はすべて文字列で、未設定・空・不正形式を起動時に検証する必要があります。Config型を一度作り、その後のコードからenv::varを直接呼ばないようにします。

この単元の到達点

- 起動時の一括読込
- 型変換
- 秘密の非表示
- 開発と本番の差

なぜバックエンドで必要か

TURSO_AUTH_TOKENやSESSION_SECRETをDebug出力しないSecret型へ入れ、起動後は検証済みConfigをAppStateで共有します。



UNIT 09・MENTAL MODEL

設定型・環境変数・秘密を図でつかむ



環境変数はすべて文字列で、未設定・空・不正形式を起動時に検証する必要があります。Config型を一度作り、その後のコードからenv::varを直接呼ばないようにします。図では左から右へ処理を追います。各箱の入口では入力 の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	起動時の一括読込	起動時の一括読込 — 入力と前提を確認し、境界を曖昧にしない。
2	型変換	型変換 — 値がどの順で変換されるかを追跡する。
3	秘密の非表示	秘密の非表示 — 失敗を再現し、型またはログから原因を特定する。
4	開発と本番の差	開発と本番の差 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

設定型・環境変数・秘密とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 09 · MINIMUM CODE

まず動く最小コード

設定型・環境変数・秘密だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
#[derive(Debug)] struct Config { host:String, port:u16 }
fn load() -> Result<Config,String> {
    let host=std::env::var("HOST").unwrap_or_else(|_| "127.0.0.1".into());
    let port=std::env::var("PORT").unwrap_or_else(|_| "3000".into()).parse().map_err(|_| "invalid PORT"?);
    Ok(Config{host,port})
}
fn main(){ println!("{:?}",load()); }
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 09 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[derive(Debug)] struct Config { host:String, port:u16 }</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>fn load() -> Result<Config,String> {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>let host=std::env::var("HOST").unwrap_or_else(_ "127.0.0.1".into());</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>let port=std::env::var("PORT").unwrap_or_else(_ "3000".into()).parse().ma</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>Ok(Config(host,port))</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力起動時の一括読込の条件を満たすか確認する。
- ✓ 型変換を実行し、途中の型と状態を追う。
- ✓ 秘密の非表示で失敗を成功経路から分離する。
- ✓ 開発と本番の差により、出力と副作用を検証する。

型を声に出す 設定型・環境変数・秘密に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 09 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
Ok(Config { host: "127.0.0.1", port: 3000 })
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 09 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
struct Secret(String);
impl std::fmt::Debug for Secret { fn fmt(&self, f:&mut std::fmt::Formatter<'_>->std::fmt::Result{f.write_str("Secret([REDACTED])")} }
impl Secret { fn expose(&self)->&str{&self.0} }
fn main(){ let s=Secret("token".into()); println!("{s:?} len={}",s.expose().len()); }
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 09・BACKEND APPLICATION

バックエンドのどこで使うか

TURSO_AUTH_TOKENやSESSION_SECRETをDebug出力しないSecret型へ入れ、起動後は検証済みConfigをAppStateで共有します。



境界で守る不変条件

- ✓ 不正な入力副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 09 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	起動時の一括読込に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	型変換の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	秘密の非表示または開発と本番の差の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 09・DEBUGGING

設定型・環境変数・秘密を再現して切り分ける

設定型・環境変数・秘密の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: 起動時の一括読み
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: 型変換
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: 秘密の非表示
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 開発と本番の差

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	起動時の一括読み	起動時の一括読み — 入力と前提を確認し、境界を曖昧にしない。
2	型変換	型変換 — 値がどの順で変換されるかを追跡する。
3	秘密の非表示	秘密の非表示 — 失敗を再現し、型またはログから原因を特定する。
4	開発と本番の差	開発と本番の差 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 設定は起動時に一度検証し、秘密は型とDebug実装で偶発的な露出を防ぎます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 09 ・ PRACTICE

手を動かす演習

Secret(String)のDebugをREDACTED表示にし、値の取得は明示的なexposeメソッドだけにしてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 09・SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
struct Secret(String);
impl std::fmt::Debug for Secret { fn fmt(&self, f: &mut std::fmt::Formatter<'_>-
>std::fmt::Result{f.write_str("Secret([REDACTED])")} } }
impl Secret { fn expose(&self)->&str{&self.0} }
fn main(){ let s=Secret("token".into()); println!("{s:?} len={}",s.expose().len()); }
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 設定は起動時に一度検証し、秘密は型とDebug実装で偶発的な露出を防ぎます。
- ✓ 設定型・環境変数・秘密とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 10

テスト・rustdoc・状態機械・APIレビュー

UNIT 10 · CONCEPT

テスト・rustdoc・状態機械・APIレビュー

設計の意図はテストとドキュメントへ固定します。状態遷移をenumとメソッドで表し、正常系、禁止遷移、境界値をテストします。公開APIには例と失敗条件を書きます。

この単元の到達点

- 単体テスト
- doc test
- 状態遷移
- 公開APIの契約

なぜバックエンドで必要か

HTTPハンドラの前に状態遷移を純粋関数で検証すると、DBやフレームワークなしで高速に設計を確認できます。



UNIT 10 · MENTAL MODEL

テスト・rustdoc・状態機械・APIレビューを図でつかむ



設計の意図はテストとドキュメントへ固定します。状態遷移をenumとメソッドで表し、正常系、禁止遷移、境界値をテストします。公開APIには例と失敗条件を書きます。図では左から右へ処理を追います。各箱の入口では入力型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	単体テスト	単体テスト — 入力と前提を確認し、境界を曖昧にしない。
2	doc test	doc test — 値がどの順で変換されるかを追跡する。
3	状態遷移	状態遷移 — 失敗を再現し、型またはログから原因を特定する。
4	公開APIの契約	公開APIの契約 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出来されるか」を自分の言葉で説明します。

30秒説明

テスト・rustdoc・状態機械・APIレビューとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 10 · MINIMUM CODE

まず動く最小コード

テスト・rustdoc・状態機械・APIレビューだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
#[derive(Debug,PartialEq)] enum State{Draft,Published,Archived}
fn publish(s:State)->Result<State,&'static str>{match s{State::Draft=>Ok(State::Published),_=>Err("invalid transition")}}
#[test] fn draft_can_publish(){assert_eq!(publish(State::Draft),Ok(State::Published));}
#[test] fn archived_cannot_publish(){assert!(publish(State::Archived).is_err());}
```

● ● ● 実行コマンド

```
cargo test
cargo doc --no-deps --open
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 10 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[derive(Debug,PartialEq)] enum State{Draft,Published,Archived}</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>fn publish(s:State)->Result<State,&'static str> {match s{State::Draft=>Ok(S</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>#[test] fn draft_can_publish(){assert_eq!(publish(State::Draft),Ok(State::</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>#[test] fn archived_cannot_publish(){assert!(publish(State::Archived).is_e</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。

実行時に起きる順序

- ✓ 入力が単体テストの条件を満たすか確認する。
- ✓ doc testを実行し、途中の型と状態を追う。
- ✓ 状態遷移で失敗を成功経路から分離する。
- ✓ 公開APIの契約により、出力と副作用を検証する。

型を声に出す テスト・rustdoc・状態機械・APIレビューに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 10 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
running 2 tests
test archived_cannot_publish ... ok
test draft_can_publish ... ok
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 10 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
#[derive(Clone, Copy, Debug, PartialEq)] enum State { Draft, Published, Archived }
fn transition(from: State, to: State) -> Result<State, &'static str> { match (from, to)
{ (State::Draft, State::Published) | (State::Published, State::Archived) => Ok(to), _ => Err("invalid") }}
#[test] fn transitions() { assert_eq!(transition(State::Draft, State::Published), Ok(State::Published)); assert!
(transition(State::Archived, State::Draft).is_err()); }
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 10 · BACKEND APPLICATION

バックエンドのどこで使うか

HTTPハンドラの前に状態遷移を純粋関数で検証すると、DBやフレームワークなしで高速に設計を確認できます。



リクエスト側

- 単体テストを入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- doc testを小さな責務へ分割
- 状態遷移をResultとログで表現
- 公開APIの契約をテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 10 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	単体テストに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	doc testの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	状態遷移または公開APIの契約の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 10・DEBUGGING

テスト・rustdoc・状態機械・APIレビューを再現して切り分ける

テスト・rustdoc・状態機械・APIレビューの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: 単体テスト
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: doc test
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: 状態遷移
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 公開APIの契約

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	単体テスト	単体テスト — 入力と前提を確認し、境界を曖昧にしない。
2	doc test	doc test — 値がどの順で変換されるかを追跡する。
3	状態遷移	状態遷移 — 失敗を再現し、型またはログから原因を特定する。
4	公開APIの契約	公開APIの契約 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 不変条件と状態遷移を型・テスト・公開ドキュメントの三か所で一致させます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 10 ・ PRACTICE

手を動かす演習

Draft→Published→Archivedのみ許すtransition関数と全組合せテストを作ってください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 10 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
#[derive(Clone, Copy, Debug, PartialEq)] enum State { Draft, Published, Archived }
fn transition(from: State, to: State) -> Result<State, &'static str> { match (from, to)
{ (State::Draft, State::Published) | (State::Published, State::Archived) => Ok(to), _ => Err("invalid") }}
#[test] fn transitions() { assert_eq!(transition(State::Draft, State::Published), Ok(State::Published)); assert!
(transition(State::Archived, State::Draft).is_err()); }
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 不変条件と状態遷移を型・テスト・公開ドキュメントの三か所で一致させます。
- ✓ テスト・rustdoc・状態機械・APIレビューとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

CAPSTONE

DB実装を交換できるNoteユースケース層を設計する

10単元を一つの成果物へ統合します。動く結果だけでなく、途中のエラー、設計判断、確認コマンドも提出物です。

順	使う単元	統合する判断
01	newtypeと不正状態を作れない型	意味と検証状態を型へ持たせ、primitive obsessionを避けます。
02	traitと境界による設計	ユースケースは必要な振る舞いをtraitで要求し、外部実装の詳細から切り離します。
03	generics・where句・静的ディスパッチ	genericは共通構造、newtypeとtraitは意味と能力の境界に使います。
04	dyn Traitとオブジェクト安全性	実装集合を実行時に選ぶ境界だけdyn Traitを使い、通常はgenericを優先します。
05	エラー型・thiserror相当の設計・文脈	エラーを原因と回復方針で分類し、HTTP表現は境界層で決めます。
06	crate・module・可視性・workspace	可視性で変更可能範囲を狭め、層の依存方向をmodule構造へ反映します。
07	Cargo features・依存固定・最小構成	featureは能力の選択であり、CIで使用する組合せをすべて検証します。
08	DTO・Serde・境界の変換	境界ごとに型を分け、未検証入力と秘密情報をドメイン・レスポンスへ持ち込みません。
09	設定型・環境変数・秘密	設定は起動時に一度検証し、秘密は型とDebug実装で偶発的な露出を防ぎます。
10	テスト・rustdoc・状態機械・APIレビュー	不変条件と状態遷移を型・テスト・公開ドキュメントの三か所で一致させます。

完了条件

- ✓ IDとメールがnewtypeである
- ✓ RepositoryとClockがtraitで差し替え可能
- ✓ AppErrorがHTTPから独立している
- ✓ 状態遷移と失敗条件にテストがある

REVIEW

巻末確認問題

用語だけでなく、「小さな例」「バックエンドでの場所」「失敗時の挙動」を含めて教えてください。

1. 01. `newtype`と不正状態を作れない型を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
2. 02. `trait`と境界による設計を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
3. 03. `generics`・`where`句・静的ディスパッチを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
4. 04. `dyn Trait`とオブジェクト安全性を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
5. 05. エラー型・`thiserror`相当の設計・文脈を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
6. 06. `crate`・`module`・可視性・`workspace`を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
7. 07. `Cargo features`・依存固定・最小構成を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
8. 08. `DTO`・`Serde`・境界の変換を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
9. 09. 設定型・環境変数・秘密を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
10. 10. テスト・`rustdoc`・状態機械・APIレビューを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

答える順序 まず資料を閉じて説明し、次に最小コードを書き、最後に該当単元へ戻って不足を補います。正解の暗記ではなく、根拠を再現できることが目標です。

REVIEW ANSWERS

確認問題・解答の骨格

次の要点は模範解答の丸暗記用ではありません。自分の説明に「定義・小さな例・バックエンドでの場所」が含まれているかを照合します。

01. newtypeと不正状態を作れない型

newtypeと不正状態を作れない型とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 意味と検証状態を型へ持たせ、primitive obsessionを避けます。

02. traitと境界による設計

traitと境界による設計とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: ユースケースは必要な振る舞いをtraitで要求し、外部実装の詳細から切り離します。

03. generics・where句・静的ディスパッチ

generics・where句・静的ディスパッチとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: genericは共通構造、newtypeとtraitは意味と能力の境界に使います。

04. dyn Traitとオブジェクト安全性

dyn Traitとオブジェクト安全性とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 実装集合を実行時に選ぶ境界だけdyn Traitを使い、通常はgenericを優先します。

05. エラー型・thiserror相当の設計・文脈

エラー型・thiserror相当の設計・文脈とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: エラーを原因と回復方針で分類し、HTTP表現は境界層で決めます。

06. crate・module・可視性・workspace

crate・module・可視性・workspaceとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 可視性で変更可能範囲を狭め、層の依存方向をmodule構造へ反映します。

07. Cargo features・依存固定・最小構成

Cargo features・依存固定・最小構成とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: featureは能力の選択であり、CIで使用する組合せをすべて検証します。

08. DTO・Serde・境界の変換

DTO・Serde・境界の変換とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 境界ごとに型を分け、未検証入力と秘密情報をドメイン・レスポンスへ持ち込みません。

09. 設定型・環境変数・秘密

設定型・環境変数・秘密とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 設定は起動時に一度検証し、秘密は型とDebug実装で偶発的な露出を防ぎます。

10. テスト・rustdoc・状態機械・APIレビュー

テスト・rustdoc・状態機械・APIレビューとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 不変条件と状態遷移を型・テスト・公開ドキュメントの三か所で一致させます。

GLOSSARY

重要用語と実務での位置

用語	一文定義	バックエンドでの位置
newtypeと不正状態を作れない型	意味と検証状態を型へ持たせ、primitive obsessionを避けます。	path parameterをNotelIdへ変換してからユースケースへ渡すと、負数やUserIdとの取り違えを境界で止められます。
traitと境界による設計	ユースケースは必要な振る舞いをtraitで要求し、外部実装の詳細から切り離します。	時刻、ID生成、メール送信、Repositoryをtrait化すると、外部状態を決定的なFakeへ置き換えられます。
generics・where句・静的ディスパッチ	genericは共通構造、newtypeとtraitは意味と能力の境界に使います。	共通のページング結果、キャッシュ、入力パーサーをgenericにしつつ、ドメイン固有の検証は専用型へ残します。
dyn Traitとオブジェクト安全性	実装集合を実行時に選ぶ境界だけdyn Traitを使い、通常はgenericを優先します。	通知チャネルや認証providerを設定で切り替える境界ではdyn Traitが有効です。hot pathではgenericとの費用差を測ります。
エラー型・thiserror相当の設計・文脈	エラーを原因と回復方針で分類し、HTTP表現は境界層で決めます。	内部エラーには原因連鎖とrequest_idを残し、レスポンスには秘密やSQLを出さず安定した分類だけを返します。
crate・module・可視性・workspace	可視性で変更可能範囲を狭め、層の依存方向をmodule構造へ反映します。	web、application、domain、infrastructureの依存方向をmodule可視性で守り、handlerからDB実装へ直接依存しない構造にします。
Cargo features・依存固定・最小構成	featureは能力の選択であり、CIで使用する組合せをすべて検証します。	本番とテストのfeature差をCIで検証し、使っていないTLSやDB driverを無効化して依存面を小さくします。
DTO・Serde・境界の変換	境界ごとに型を分け、未検証入力と秘密情報をドメイン・レスポンスへ持ち込みません。	Web入力DTO、検証済みドメイン型、DB Row、レスポンスDTOを分けることで、未検証値と秘密フィールドの層越えを防ぎます。
設定型・環境変数・秘密	設定は起動時に一度検証し、秘密は型とDebug実装で偶発的な露出を防ぎます。	TURSO_AUTH_TOKENやSESSION_SECRETをDebug出力しないSecret型へ入れ、起動後は検証済みConfigをAppStateで共有します。
テスト・rustdoc・状態機械・APIレビュー	不変条件と状態遷移を型・テスト・公開ドキュメントの三か所で一致させます。	HTTPハンドラの前に状態遷移を純粋関数で検証すると、DBやフレームワークなしで高速に設計を確認できます。

PRIMARY SOURCES

公式資料と更新確認

仕様は更新されます。教材で意図を理解した後、実装時点の一次資料でAPI、security note、breaking changeを確認してください。

1. Rust Book

<https://doc.rust-lang.org/stable/book/>
Rust 2024 Edition を前提にした公式入門書

2. Rust Reference

<https://doc.rust-lang.org/reference/>
言語仕様を確認する一次資料

3. Standard Library

<https://doc.rust-lang.org/std/>
標準ライブラリAPI

4. Cargo Book

<https://doc.rust-lang.org/cargo/>
ビルド、依存関係、workspace、features

5. Rustdoc Book

<https://doc.rust-lang.org/rustdoc/>
APIドキュメントの書き方

6. Clippy

<https://doc.rust-lang.org/clippy/>
Rust公式Lint

7. Async Book

<https://rust-lang.github.io/async-book/>
非同期Rustの公式解説

8. Tokio Tutorial

<https://tokio.rs/tokio/tutorial>
Tokio公式チュートリアル

9. Topcoat

<https://github.com/tokio-rs/topcoat>
Topcoat公式リポジトリ。early-stageの注意を含む

10. Topcoat 0.5.0 API

<https://docs.rs/topcoat/0.5.0/topcoat/>
本教材で固定するAPI

11. Turso Rust SDK

<https://docs.turso.tech/sdk/rust/quickstart>
libSQL Rust SDK公式手順

12. Better Auth Database

<https://better-auth.com/docs/concepts/database>
Better AuthのDB構成

13. Fly.io Deploy

<https://fly.io/docs/launch/deploy/>
fly launch / fly deploy公式手順

14. Fly.io Health Checks

<https://fly.io/docs/reference/health-checks/>
公開後のヘルスチェック

15. OWASP Cheat Sheets

<https://cheatsheetseries.owasp.org/>
Webセキュリティの実務チェック

採用判断 本教材はRust 2024 Edition、Topcoat 0.5.0固定、Turso/libSQL、Topcoat Session + Argon2id、Fly.ioを主経路にします。Better AuthはTypeScript sidecarが適切な要件で比較します。